

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук Образовательная программа «Программная инженерия»

СОГЛАСОВАНО	УТВЕРЖДАЮ
Научный руководитель, ассистент департамента программной инженерии	Академический руководитель образовательной программы «Программная инженерия» старший преподаватель департамента программной инженерии
_____ В. С. Смолин	_____ Н. А. Павлов
«__» _____ 2025 г.	«__» _____ 2025 г.

Фамильный Круг — сервис совместного создания и визуализации семейного древа

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.01-01 81 01-1-ЛУ

Исполнитель студент группы БПИ-235 _____ / Д. Е. Баженов / «__» _____ 2025 г.

Москва 2025

УТВЕРЖДЁН RU.17701729.05.01-01 81 01-1-ЛУ

Фамильный Круг — сервис совместного создания и визуализации семейного древа

Пояснительная записка

RU.17701729.05.01-01 81 01-1

Листов 20

Москва 2025

ОГЛАВЛЕНИЕ

- [АННОТАЦИЯ](#)
- [1. ВВЕДЕНИЕ](#)
- [2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ](#)

- 3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ
 - 3.1. Постановка задачи
 - 3.2. Описание применяемых алгоритмов
 - 3.3. Описание структуры программы и технических средств
 - 3.4. Обоснование принятых технических решений
 - 3.5. Описание организации входных и выходных данных
 - 4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ
 - 5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ
 - ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ
-

АННОТАЦИЯ

Настоящий документ является пояснительной запиской к программному средству «Фамильный Круг — сервис совместного создания и визуализации семейного древа» и разработан в соответствии с требованиями ГОСТ 19.404-79 «Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению».

В документе изложены назначение, область применения, технические характеристики, описание применяемых алгоритмов и архитектурных решений, обоснование выбора технологического стека, а также ожидаемые показатели программного средства.

Программа разработана на языках Java 17 (серверная часть) и JavaScript/JSX (клиентская часть). Серверная часть реализована на платформе Spring Boot, клиентская — на основе библиотеки React 19 с использованием сборщика Vite.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 — Виды программ и программных документов.
2. ГОСТ 19.102-77 — Стадии разработки.
3. ГОСТ 19.103-77 — Обозначения программ и программных документов.
4. ГОСТ 19.104-78 — Основные надписи.
5. ГОСТ 19.105-78 — Общие требования к программным документам.
6. ГОСТ 19.106-78 — Требования к программным документам, выполненным печатным способом.
7. ГОСТ 19.404-79 — Пояснительная записка. Требования к содержанию и оформлению.

Изменения к данному документу оформляются согласно ГОСТ 19.603-78 и ГОСТ 19.604-78.

Ссылка на репозиторий программного средства: <https://github.com/bazhenov-denis/family-tree>

1. ВВЕДЕНИЕ

Настоящая пояснительная записка описывает программное средство «Фамильный Круг» — веб-приложение для совместного создания, редактирования и визуализации семейных деревьев. Документ содержит обоснование принятых проектных решений, описание архитектуры и алгоритмов, а также сведения об организации входных и выходных данных.

Разработка выполнена студентом группы БПИ-235 Баженовым Денисом Евгеньевичем в рамках учебного проекта на факультете компьютерных наук НИУ ВШЭ.

Программное средство находится в открытом репозитории: <https://github.com/bazhenov-denis/family-tree>

Структура репозитория:

Директория	Содержание
<code>backend/</code>	Серверная часть (Spring Boot, Java 17)
<code>frontend/</code>	Клиентская часть (React 19, Vite)
<code>nginx/</code>	Конфигурация обратного прокси
<code>docker-compose.yml</code>	Файл оркестрации контейнеров
<code>docs/</code>	Программная документация

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

Назначение программы. «Фамильный Круг» — веб-сервис для создания, хранения и визуализации родословных. Программа позволяет пользователям формировать семейные деревья, вносить биографические сведения о родственниках, фиксировать жизненные события, прикреплять медиаконтент (фотографии, документы), а также совместно работать над деревом в многопользовательском режиме с разграничением прав доступа.

Область применения. Программное средство предназначено для:

- частных пользователей, занимающихся составлением родословных и сохранением семейной памяти;
- исследователей, ведущих генеалогическую работу;
- коллективов (семей, исторических сообществ), нуждающихся в совместном редактировании и хранении генеалогических данных.

Работа с программой осуществляется через веб-браузер без установки дополнительного программного обеспечения на компьютер конечного пользователя.

Основные функциональные возможности:

№	Группа функций	Перечень
1	Управление учётными записями	Регистрация, вход, восстановление сессии, редактирование профиля, смена аватара
2	Управление деревьями	Создание, редактирование, удаление деревьев; многодеревная рабочая среда
3	Управление персонами	Добавление, редактирование, удаление записей о людях
4	Родственные связи	Установление связей типов «родитель», «супруг», «опекун», «усыновление»

№	Группа функций	Перечень
5	Визуализация	SVG-граф, режим «Веер», режим «Хронология», сворачивание ветвей
6	Поиск и анализ	Поиск по имени, вычисление пути родства, статистика дерева
7	События и медиа	Жизненные события персон, фотогалерея, прикрепление документов
8	Комментарии	Комментирование персон и событий
9	Импорт / Экспорт	Экспорт в GEDCOM и JSON; импорт из GEDCOM с предварительным просмотром
10	Многопользовательская работа	Роли доступа, приглашения по токен-ссылке, журнал изменений
11	Пользовательский опыт	Тёмная тема, ближайшие дни рождения, адаптивная навигация

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи

Перед разработкой была сформулирована следующая задача: создать веб-приложение, которое обеспечивает:

- Хранение генеалогических данных** — структурированная база данных для хранения сведений о людях, их родственных связях, жизненных событиях и медиаконтенте.
- Визуализацию родословной** — интерактивный граф семейного дерева с возможностью навигации (масштабирование, перетаскивание), поиска и сворачивания ветвей.
- Многопользовательский доступ** — система ролей (владелец, редактор, наблюдатель, комментатор) и механизм приглашений для совместной работы над деревом.
- Совместимость с отраслевым стандартом** — поддержка формата GEDCOM 5.5.1 для обмена данными с другими генеалогическими системами.
- Безопасность и аутентификацию** — stateless JWT-аутентификация без хранения серверных сессий.
- Масштабируемость развёртывания** — контейнеризация через Docker Compose для единообразного запуска всего стека.

Ограничения и допущения:

- Интерфейс программы реализован только на русском языке.
- Генерация схемы базы данных выполняется автоматически средствами Hibernate (`ddl-auto: update`); Flyway-миграции отключены.

- Реализован режим MVP: функции совместного редактирования в реальном времени через WebSocket в данной версии отсутствуют; история изменений ведётся через аудит-лог.

3.2. Описание применяемых алгоритмов

3.2.1. Алгоритм вычисления пути родства (BFS)

Для нахождения цепочки родственных связей между двумя персонами используется алгоритм обхода графа в ширину (Breadth-First Search, BFS), реализованный на стороне клиента в файле `frontend/src/utils/findRelationPath.js`.

Описание алгоритма:

1. Построить граф смежности: для каждой персоны собрать список её непосредственных соседей (родители, дети, супруги, опекуны).
2. Инициализировать очередь с начальной персоной; отметить её как посещённую.
3. На каждой итерации извлекать персону из очереди и проверять всех её соседей:
 - если сосед является целевой персоной — восстановить путь через массив предков и завершить поиск;
 - если сосед не посещён — добавить в очередь и отметить посещённым.
4. Если очередь опустела, а цель не достигнута — вернуть «путь не найден».

Сложность: $O(V + E)$, где V — количество персон, E — количество родственных связей в дереве.

3.2.2. Алгоритм расстановки узлов графа (Genealogy Layout)

Для автоматического позиционирования карточек персон в SVG-пространстве реализован собственный алгоритм иерархической расстановки, описанный в файле `frontend/src/utils/genealogyLayout.js`.

Параметры расстановки:

Константа	Значение	Назначение
CARD_W	155 px	Ширина карточки персоны
CARD_H	180 px	Высота карточки персоны
H_GAP	60 px	Горизонтальный зазор между карточками
COUPLE_GAP	28 px	Зазор между карточками супругов
V_GAP	80 px	Вертикальный зазор между поколениями

Описание алгоритма:

1. Определить корневые узлы дерева (персоны без родителей в системе).
2. Обойти граф в глубину (DFS), присвоив каждой персоне уровень поколения.
3. Сгруппировать супругов: пара размещается рядом с зазором `COUPLE_GAP`; их общие дети центрируются под парой.
4. Разрешить конфликты позиций: если ветви перекрываются по горизонтали, сдвинуть правую ветвь на необходимый отступ.

5. Рассчитать абсолютные координаты (x, y) каждого узла на основании уровня поколения и горизонтальной позиции в ряду.

Режим «Веер»: персоны располагаются радиально относительно выбранного центрального предка; угол каждого узла рассчитывается пропорционально его позиции в ряду поколения.

Режим «Хронология»: персоны упорядочиваются по оси X согласно году рождения; ось Y фиксирована по поколению.

3.2.3. Алгоритм сворачивания ветвей

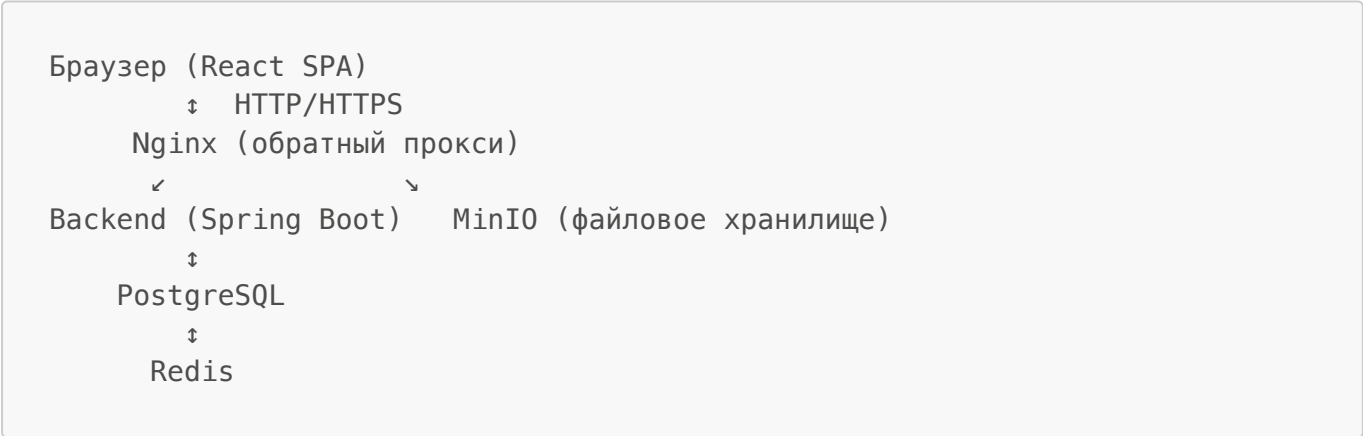
Скрытие дочерних узлов реализовано функцией `computeHidden()` в `frontend/src/components/tree/TreeGraph.jsx`:

- 1. Хранить множество `collapsedIds` — идентификаторы персон, для которых пользователь нажал кнопку сворачивания.
- 2. При каждом рендере для каждого узла из `collapsedIds` рекурсивно собирать все дочерние идентификаторы в множество `hiddenIds`.
- 3. При отрисовке SVG пропускать узлы и рёбра, идентификатор которых входит в `hiddenIds`.

3.3. Описание структуры программы и технических средств

3.3.1. Общая архитектура

Программа реализована по клиент-серверной трёхзвенной архитектуре:



Все компоненты упакованы в Docker-образы и запускаются через `docker-compose.yml`.

3.3.2. Серверная часть (Backend)

Платформа: Java 17, Spring Boot 3.x
Корневой пакет: `com.example.backend`

Серверная часть организована по доменному принципу. Каждый домен содержит слои:

Слой	Пакет	Назначение
HTTP-слой	<code>api/</code>	<code>@RestController</code> — приём и возврат HTTP-запросов
Бизнес-логика	<code>application/</code>	<code>@Service</code> — обработка данных, вызов репозиториев

Слой	Пакет	Назначение
Сущности	<code>entity/</code>	JPA <code>@Entity</code> — ORM-маппинг таблиц БД
DTO	<code>dto/</code>	Объекты для сериализации запросов и ответов
Репозитории	<code>repository/</code>	Spring Data JPA — запросы к базе данных

Доменные модули:

Модуль	Ключевые сущности	Назначение
<code>auth</code>	<code>User</code>	JWT-аутентификация, регистрация, профиль
<code>person</code>	<code>Person</code>	CRUD персон семейного дерева
<code>relationship</code>	<code>Relationship</code>	Родственные связи четырёх типов
<code>event</code>	<code>Event</code> , <code>EventPerson</code>	Жизненные события персон
<code>comment</code>	<code>Comment</code>	Комментарии к любой записи
<code>tree</code>	<code>FamilyTree</code> , <code>TreeMember</code>	Деревья и участники с ролями
<code>tree/invite</code>	<code>InviteToken</code> , <code>TreeInvite</code>	Токен-приглашения по электронной почте
<code>media</code>	<code>Media</code>	Загрузка и хранение медиафайлов через MinIO
<code>graph</code>	—	Вспомогательные алгоритмы визуализации (серверная часть)
<code>gedcom</code>	—	Парсер и генератор GEDCOM 5.5.1
<code>audit</code>	<code>AuditLog</code>	Журнал изменений
<code>shared</code>	<code>BaseEntity</code>	Глобальные исключения, конфиги MinIO и почты

Безопасность:

- `JwtTokenProvider` — генерация и валидация JWT.
- `JwtAuthenticationFilter` — фильтр Spring Security, добавляемый перед стандартной цепочкой.
- Публичные эндпоинты: `/api/auth/**`, `/api/ping`, `/api/invites/**`.
- Все остальные маршруты требуют действительного Bearer-токена.

База данных: PostgreSQL 15. Схема создаётся автоматически Hibernate (`ddl-auto: update`). Соединение через Spring Data JPA / Hibernate ORM.

Кеш: Redis 7 — используется для хранения вспомогательных данных и сессий токенов.

Файловое хранилище: MinIO (S3-совместимый объектный стор). Файлы сохраняются в бакет `family-tree-media`; URL для доступа проксируется через Nginx по пути `/media/`.

3.3.3. Клиентская часть (Frontend)

Стек: React 19, Vite 7, React Router 7, plain JSX (без TypeScript в исходных файлах).
Стилизация: собственный CSS в frontend/src/index.css (~1800 строк, CSS-переменные, поддержка тёмной темы).

Маршруты приложения:

Путь	Компонент	Описание
/	HomeRedirect	Перенаправление: /trees или лендинг
/login	LoginPage	Страница входа
/register	RegisterPage	Страница регистрации
/invite/:token	InviteAcceptPage	Принятие приглашения
/trees	TreesPage	Список деревьев пользователя
/trees/:treeId	TreeDetailPage	Граф дерева
/trees/:treeId/members	TreeMembersPage	Участники дерева
/trees/:treeId/persons	PersonsPage	Список персон
/invites	InvitesPage	Отправленные приглашения
/profile	ProfilePage	Профиль пользователя

Ключевые компоненты:

Компонент	Файл	Описание
Layout	components/Layout.jsx	Боковая навигация, тёмная тема, мобильный drawer
TreeGraph	components/tree/TreeGraph.jsx	SVG-граф дерева, интерактивность
TreeSearch	components/tree/TreeSearch.jsx	Поиск персон в дереве
PersonPanel	components/person/PersonPanel.jsx	4-вкладочная панель персоны
MediaGallery	components/person/MediaGallery.jsx	Галерея фото и документов
CommentSection	components/person/CommentSection.jsx	Комментарии к записи
Modal	components/ui/Modal.jsx	Базовый модальный диалог
ConfirmModal	components/ui/ConfirmModal.jsx	Диалог подтверждения операции
ToastContainer	components/ui/ToastContainer.jsx	Всплывающие уведомления

API-слой: каждый домен имеет отдельный JS-модуль в frontend/src/api/ (например, trees.js, persons.js). Базовый клиент api/client.js добавляет заголовок Authorization: Bearer <token> и обрабатывает ответы 401 (перенаправление на /login).

3.3.4. Инфраструктура и развёртывание

Nginx выступает единой точкой входа и проксирует запросы:

Путь	Назначение
/	Клиентская часть (React SPA)
/api/	Серверная часть (Spring Boot, порт 8080)
/media/	MinIO (порт 9000)

Docker Compose описывает 6 сервисов:

Сервис	Образ	Внешний порт
postgres	postgres:15	5434
redis	redis:7	6379
minio	minio/minio	9000 (API), 9001 (консоль)
backend	(собственный образ)	— (внутренний 8080)
frontend	(собственный образ)	— (внутренний 3000)
nginx	nginx:1.25	80

3.4. Обоснование принятых технических решений

3.4.1. Выбор языка и фреймворка серверной части

Java 17 + Spring Boot 3 выбраны по следующим причинам:

- зрелая экосистема с широким набором готовых интеграций (Spring Security, Spring Data JPA, Spring Mail);
- строгая типизация и встроенные инструменты рефакторинга снижают вероятность ошибок;
- поддержка аннотаций Lombok исключает необходимость написания шаблонного кода (геттеры, сеттеры, конструкторы);
- отраслевой стандарт для enterprise-разработки, хорошо знакомый целевой аудитории (студенты программной инженерии).

3.4.2. Выбор СУБД

PostgreSQL 15 выбран как основная СУБД по следующим причинам:

- надёжная поддержка транзакций (ACID);
- возможность хранения структурированных отношений «многие ко многим» (родственные связи, участники дерева);
- встроенная поддержка JSONB для хранения произвольных атрибутов;
- хорошая интеграция с Hibernate/Spring Data JPA.

Redis используется как вспомогательный кеш: высокая скорость чтения/записи в памяти подходит для хранения краткосрочных данных.

3.4.3. Выбор объектного хранилища

MinIO выбран вместо хранения файлов в СУБД по следующим причинам:

- хранение бинарных файлов в реляционной БД снижает производительность;
- S3-совместимый API даёт возможность в будущем перейти на облачный хостинг (AWS S3, Yandex Object Storage) без изменения кода;
- MinIO легко развёртывается локально в Docker-контейнере.

3.4.4. Выбор технологий клиентской части

React 19 выбран как основная UI-библиотека:

- компонентный подход упрощает переиспользование элементов интерфейса;
- React Router 7 обеспечивает клиентскую маршрутизацию (SPA) без перезагрузки страницы;
- большое сообщество и богатая экосистема пакетов.

Vite 7 используется как инструмент сборки:

- значительно быстрее Webpack при разработке (HMR на основе ESM);
- минимальная конфигурация «из коробки».

Plain JSX (без TypeScript) — принципиальный выбор для снижения порога вхождения и ускорения разработки MVP. TypeScript-зависимости присутствуют в `devDependencies` для возможной миграции в будущем.

SVG-граф реализован без сторонних графовых библиотек (D3.js, vis-network) — это исключает избыточные зависимости и обеспечивает полный контроль над отрисовкой. Расчёт координат узлов выполняется собственным алгоритмом (`genealogyLayout.js`).

CSS без фреймворков (без Tailwind, styled-components) — единый файл `index.css` (~1800 строк) с CSS-переменными даёт полный контроль над стилями и поддержкой тёмной темы без дополнительных зависимостей.

3.4.5. Выбор формата обмена данными

GEDCOM 5.5.1 — международный открытый стандарт генеалогических данных, поддерживаемый всеми крупными генеалогическими платформами (Ancestry, FamilySearch, MyHeritage). Поддержка GEDCOM обеспечивает совместимость и возможность миграции данных.

JSON-экспорт — собственный формат, включающий полный снимок дерева (персоны, связи, события, медиа) для резервного копирования и интеграции со сторонними инструментами.

3.4.6. Аутентификация

Stateless JWT выбран вместо серверных сессий:

- не требует хранилища сессий (горизонтальное масштабирование упрощается);

- токен содержит всю необходимую информацию о пользователе (subject, expiration);
- стандартный подход для REST API.

3.4.7. Контейнеризация

Docker Compose обеспечивает воспроизводимую среду: одна команда `docker compose up` разворачивает весь стек (6 сервисов) в согласованном состоянии. Это исключает расхождения «работает на моей машине» и упрощает проверку работоспособности.

3.5. Описание организации входных и выходных данных

3.5.1. Входные данные

Входные данные поступают в систему из следующих источников:

1. HTTP-запросы от клиентской части

Все запросы направляются на `/api/**`. Тело запросов передаётся в формате JSON; файлы — в формате `multipart/form-data`.

Тип данных	Формат	Пример эндпоинта
Учётные данные	JSON <code>{email, password, name}</code>	POST <code>/api/auth/login</code>
Данные персоны	JSON <code>{firstName, lastName, gender, birthDate, ...}</code>	POST <code>/api/trees/{id}/persons</code>
Родственная связь	JSON <code>{fromPersonId, toPersonId, type}</code>	POST <code>/api/relationships</code>
Жизненное событие	JSON <code>{type, date, description}</code>	POST <code>/api/events</code>
Комментарий	JSON <code>{entityType, entityId, text}</code>	POST <code>/api/comments</code>
Медиафайл	<code>multipart/form-data</code> (файл + метаданные)	POST <code>/api/upload/photo</code>
GEDCOM-файл	<code>multipart/form-data</code> (текстовый файл <code>.ged</code>)	POST <code>/api/gedcom/import</code>
Данные приглашения	JSON <code>{email, role, treeId}</code>	POST <code>/api/invites</code>

2. Пользовательский ввод в форме браузера

Данные вводятся через HTML-формы в интерфейсе и передаются клиентской частью в виде JSON через API-слой.

3. Внешние файлы

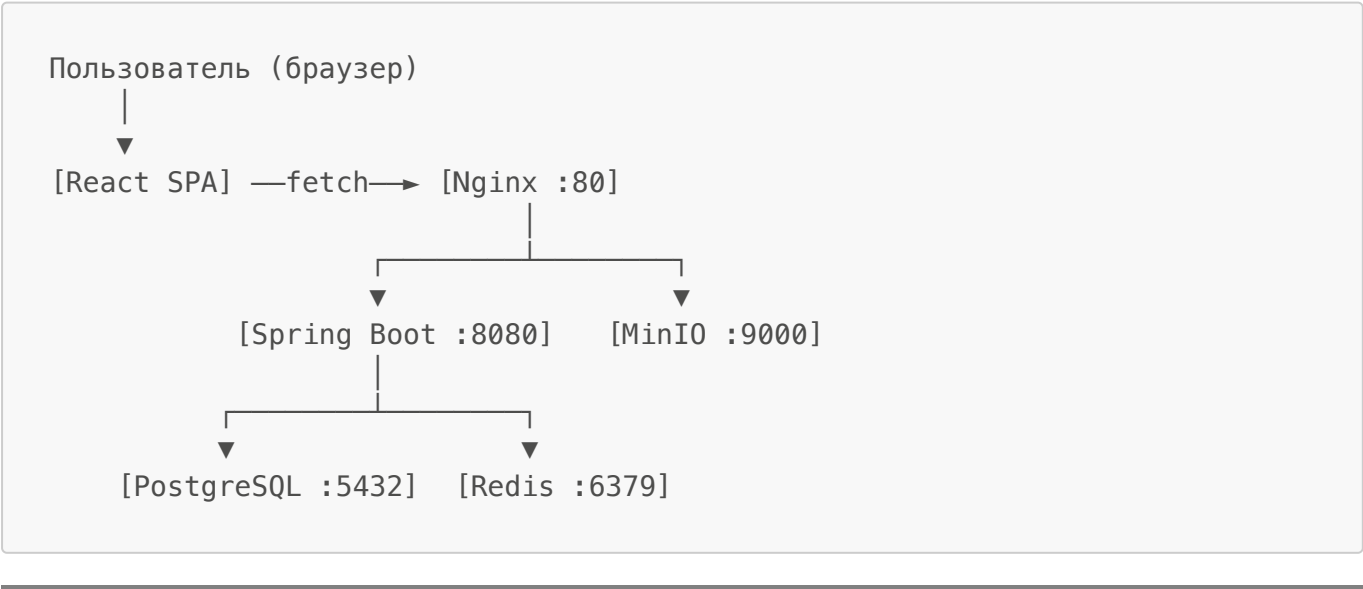
- Файлы GEDCOM (`.ged`) — стандарт 5.5.1, кодировка UTF-8 или ANSEL.

- Медиафайлы: фотографии (JPEG, PNG, WEBP ≤ 10 МБ), документы (PDF, DOC, DOCX, XLS, XLSX, TXT ≤ 10 МБ).

3.5.2. Выходные данные

Тип выходных данных	Формат	Описание
JWT-токен	Строка (Bearer)	Возвращается при успешной аутентификации
Список деревьев	JSON-массив	Отображается на странице «Мои деревья»
Данные персоны	JSON-объект	Отображаются в панели персоны
SVG-граф	SVG-разметка	Визуализация родственных связей в браузере
Путь родства	JSON-массив шагов	Преобразуется в текстовое описание на клиенте
Статистика дерева	JSON-объект	Отображается в модальном диалоге
GEDCOM-файл	Текстовый файл .ged	Скачивается в браузере при экспорте
JSON-файл дерева	Текстовый файл .json	Скачивается в браузере при экспорте
Аудит-лог	JSON-массив записей	Отображается в диалоге «Журнал изменений»
URL медиафайла	Строка	Используется для отображения изображений через MinIO
Сообщение об ошибке	JSON {message} / Toast	Отображается при нарушении ограничений

3.5.3. Схема потоков данных



4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Функциональная полнота

По результатам разработки реализованы все 39 заявленных в техническом задании функций (Ф-01 — Ф-39). Процент выполнения функциональных требований составляет **100%**.

Группа функций	Запланировано	Реализовано
Аутентификация и профиль	5	5
Управление деревьями	4	4
Персоны и связи	7	7
Визуализация	5	5
Поиск и анализ	4	4
События	3	3
Медиаконтент	3	3
Комментарии	2	2
Импорт / Экспорт	3	3
Участники и приглашения	4	4
Прочее	1	1
Итого	41	41

4.2. Технические показатели

Показатель	Значение
Время первоначальной загрузки приложения	≤ 3 с (при скорости соединения ≥ 10 Мбит/с)
Время отрисовки графа (до 50 персон)	≤ 1 с
Время отклика REST API (типовой запрос)	≤ 200 мс
Максимальный размер загружаемого файла	10 МБ
Поддерживаемые браузеры	Chrome ≥ 110, Firefox ≥ 110, Edge ≥ 110
Минимальное разрешение экрана	1280 × 720 пикселей
Количество одновременно поддерживаемых деревьев на аккаунт	Не ограничено

4.3. Экономические показатели

Разработка выполнена в рамках учебного проекта одним исполнителем. Экономический эффект от внедрения выражается в:

- **сокращении времени** на составление и оформление родословных за счёт автоматической SVG-визуализации и поддержки стандарта GEDCOM (импорт из сторонних систем);
- **снижении порога вхождения** в генеалогическую работу за счёт интуитивного веб-интерфейса на русском языке;
- **повышении сохранности данных** за счёт централизованного хранилища с резервным экспортом в GEDCOM и JSON.

5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

1. ГОСТ 19.404-79. Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению. — М.: Стандартиформ, 2010.
2. ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов. — М.: Стандартиформ, 2010.
3. ГОСТ 19.102-77. Единая система программной документации. Стадии разработки. — М.: Стандартиформ, 2010.
4. ГОСТ 19.103-77. Единая система программной документации. Обозначения программ и программных документов. — М.: Стандартиформ, 2010.
5. ГОСТ 19.105-78. Единая система программной документации. Общие требования к программным документам. — М.: Стандартиформ, 2010.
6. ГОСТ 19.603-78. Единая система программной документации. Общие правила внесения изменений. — М.: Стандартиформ, 2010.
7. ГОСТ 19.604-78. Единая система программной документации. Правила внесения изменений в программные документы, выполненные печатным способом. — М.: Стандартиформ, 2010.
8. Spring Boot Reference Documentation. Электронный ресурс. URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (дата обращения 28.04.2025).
9. React Documentation. Электронный ресурс. URL: <https://react.dev/> (дата обращения 28.04.2025).
10. Vite Documentation. Электронный ресурс. URL: <https://vitejs.dev/guide/> (дата обращения 28.04.2025).
11. Spring Security Reference. Электронный ресурс. URL: <https://docs.spring.io/spring-security/reference/> (дата обращения 28.04.2025).
12. PostgreSQL 15 Documentation. Электронный ресурс. URL: <https://www.postgresql.org/docs/15/> (дата обращения 28.04.2025).
13. MinIO Documentation. Электронный ресурс. URL: <https://min.io/docs/> (дата обращения 28.04.2025).
14. Redis Documentation. Электронный ресурс. URL: <https://redis.io/docs/> (дата обращения 28.04.2025).
15. GEDCOM Standard Release 5.5.1. — The Church of Jesus Christ of Latter-day Saints, 1999.
16. RFC 7519. JSON Web Token (JWT). — IETF, 2015.
17. Docker Documentation. Электронный ресурс. URL: <https://docs.docker.com/> (дата обращения 28.04.2025).
18. Cormen T.H., Leiserson C.E., Rivest R.L., Stein C. Introduction to Algorithms. 4th ed. — MIT Press, 2022. (Алгоритмы BFS — гл. 22.)
19. React Router Documentation. Электронный ресурс. URL: <https://reactrouter.com/en/main> (дата обращения 28.04.2025).

20. Репозиторий проекта «Фамильный Круг». Электронный ресурс. URL:
<https://github.com/bazhenov-denis/family-tree> (дата обращения 28.04.2025).

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Изм.	Номера листов (страниц)	Всего листов (страниц) в документе	№ документа	Входящий № сопроводительного документа и дата	Подпись	Дата
	Изменённых	Заменённых	Новых	Аннулированных		